

Input Validation in C

Filtering Reality for Software Security

A Student's Guide to Writing Secure Code

Author: CodeHive Educational Labs

Release Date: February 2026

1. What is Input Validation?

In the world of C programming, **Input Validation** is the first line of defence. It is the process where a program verifies that the data provided by a user (or another program) matches the expected format, type, and range.

Because C is a low-level language, it does not automatically stop you from putting a 50-letter name into a 10-letter box. Without validation, your program is vulnerable to crashes and hackers.

The Golden Rule: Never trust user input. Assume every user will try to break your program, either by accident or on purpose.

2. Why Validation is Critical

- **Prevents Buffer Overflows:** Stopping data from spilling into memory where it doesn't belong.
- **Avoids Crashes:** Ensuring you don't perform math on letters or divide by zero.
- **Data Integrity:** Making sure a "Price" variable isn't a negative number.
- **User Experience:** Providing helpful error messages instead of a silent crash.

CODEHIVE

3. Validating with scanf()

The most common beginner mistake is ignoring the return value of `scanf()`. This function returns the number of items successfully mapped to variables.

```
int age;
printf("Enter your age: ");
if (scanf("%d", &age) != 1) {
    printf("Error: That is not a valid number!\n");
    // Clear the input buffer code here...
}
```

4. Cleaning the Input Buffer

When `scanf` fails, the "bad" input stays in the buffer. If you don't clean it, your next `scanf` will immediately fail too, often causing an infinite loop.

The "Buffer Flush" Trick:

```
int c;  
while ((c = getchar()) != '\n' && c != EOF);  
// This clears the 'trash' characters from the keyboard buffer.
```

5. Range Checking

Even if the input is a number, it might not be a *logical* number. A program for a cinema shouldn't accept an age of -5 or 200.

```
int score;
scanf("%d", &score);
if (score < 0 || score > 100) {
    printf("Invalid score! Must be between 0-100.\n");
}
```

6. Validating Strings: fgets() vs gets()

DANGER: Never use `gets ( )`. It was removed from the C standard because it cannot check how big your array is, making it a massive security risk.

Always use `fgets ( )` because it allows you to specify the maximum characters to read.

```
char name[20];  
printf("Enter name: ");  
fgets(name, sizeof(name), stdin);
```

7. Loop Until Correct

A professional program doesn't just quit when a user makes a typo; it asks again.

```
int quantity;
while (1) {
    printf("Enter quantity (1-10): ");
    if (scanf("%d", &quantity) == 1 && quantity >= 1 && quantity <=
10) {
        break; // Input is perfect!
    }
    printf("Try again! ");
    while (getchar() != '\n'); // Clear buffer
}
```

8. Advanced: strtol() for Numbers

For high-security applications, using `strtol()` is better than `scanf` because it can tell the difference between "0" and "ABC" (which `atoi` cannot).

```
char input[10];
fgets(input, 10, stdin);
char *endptr;
long val = strtol(input, &endptr, 10);
if (input == endptr) printf("No digits found.");
```

9. Validating Specific Characters

Common for menu selections (Y/N, A/B/C).

```
char choice;
printf("Continue? (y/n): ");
scanf(" %c", &choice); // Note the space before %c
if (choice != 'y' && choice != 'n') {
    printf("Invalid selection.\n");
}
```

10. File Input Validation

Files can be corrupted or mistakenly edited. Always validate data read from files just as strictly as keyboard input.

```
if (fscanf(filePtr, "%d", &data) != 1) {  
    printf("File format error!\n");  
}
```

11. Common Validation Mistakes

- **Assuming strings have no spaces:** `scanf ( "%s" )` stops at the first space.
- **Forgetting the Null Terminator:** Budgeting for 10 chars in a 10-char array (need 11!).
- **Leaving the newline:** `fgets` keeps the `'\n'` at the end of the string.

CODEHIVE

12. Summary Checklist

1. Is the data the right type? (Int vs Char)
2. Is the data in the right range? (0 to 100)
3. Is the data the right size? (String length)
4. Did I clear the buffer on failure?

CODEHIVE

End of Module

You have mastered the basics of Input Validation.

CODEHIVE STUDENT SERIES

© 2026 CodeHive Edu. All rights reserved.